

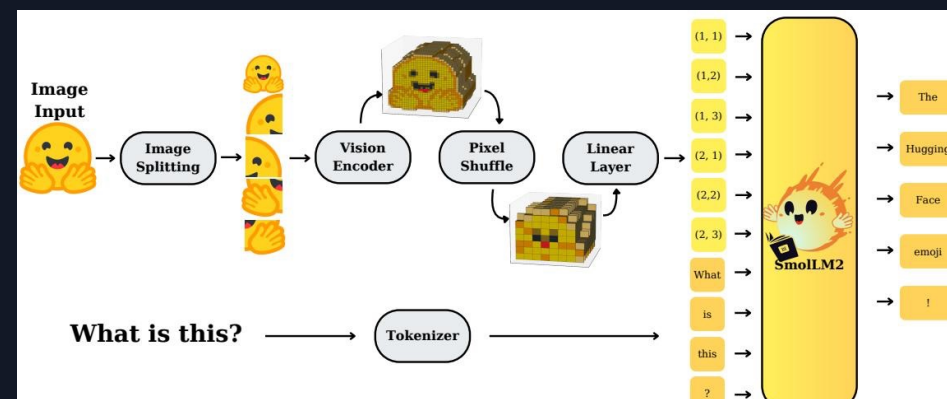
SmolVLM: Redefining Small and Efficient Multimodal Models

Hugging Face & Stanford • Efficient on-device VLMs

Key idea

Compact VLM family: 256M / 500M / 2.2B parameters
Architectural efficiency > raw scale for edge deployment
Sub-1GB inference (0.8GB at batch=1) is achievable
Open-sourced weights, data, code + HuggingSnap iOS app

Architecture at a glance



Large VLMs Are Powerful but Impractical for Edge Deployment

Why edge deployment is hard

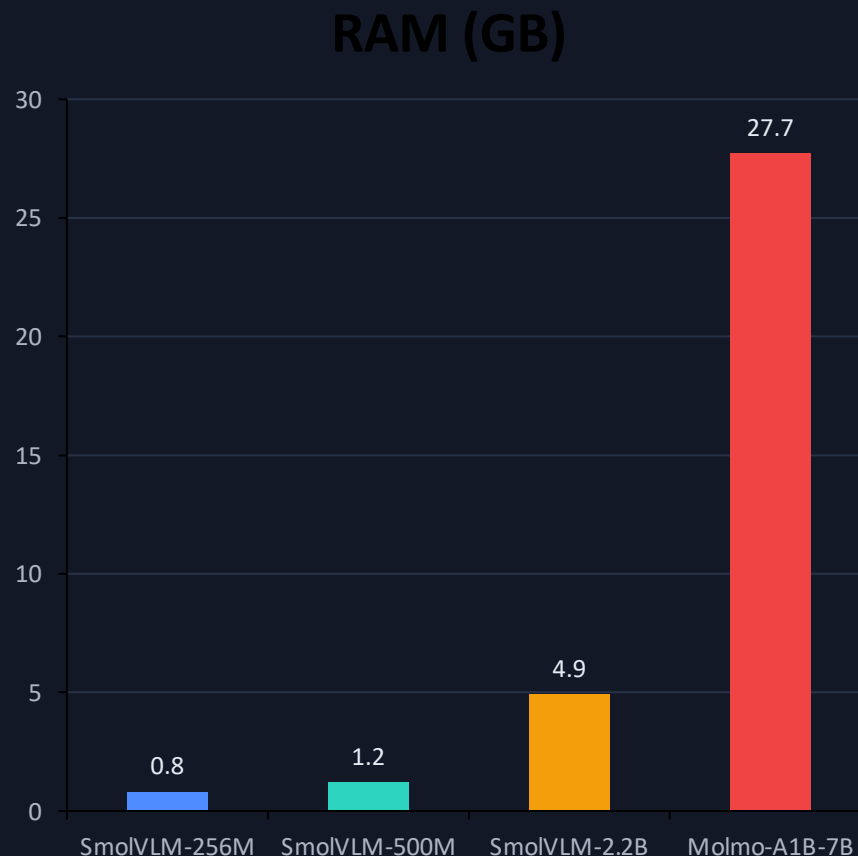
Memory dominates: attention grows with visual tokens + context length

Many strong open-source baselines require tens of GB RAM

Edge constraints: <1–5GB memory budgets, low power, real-time latency

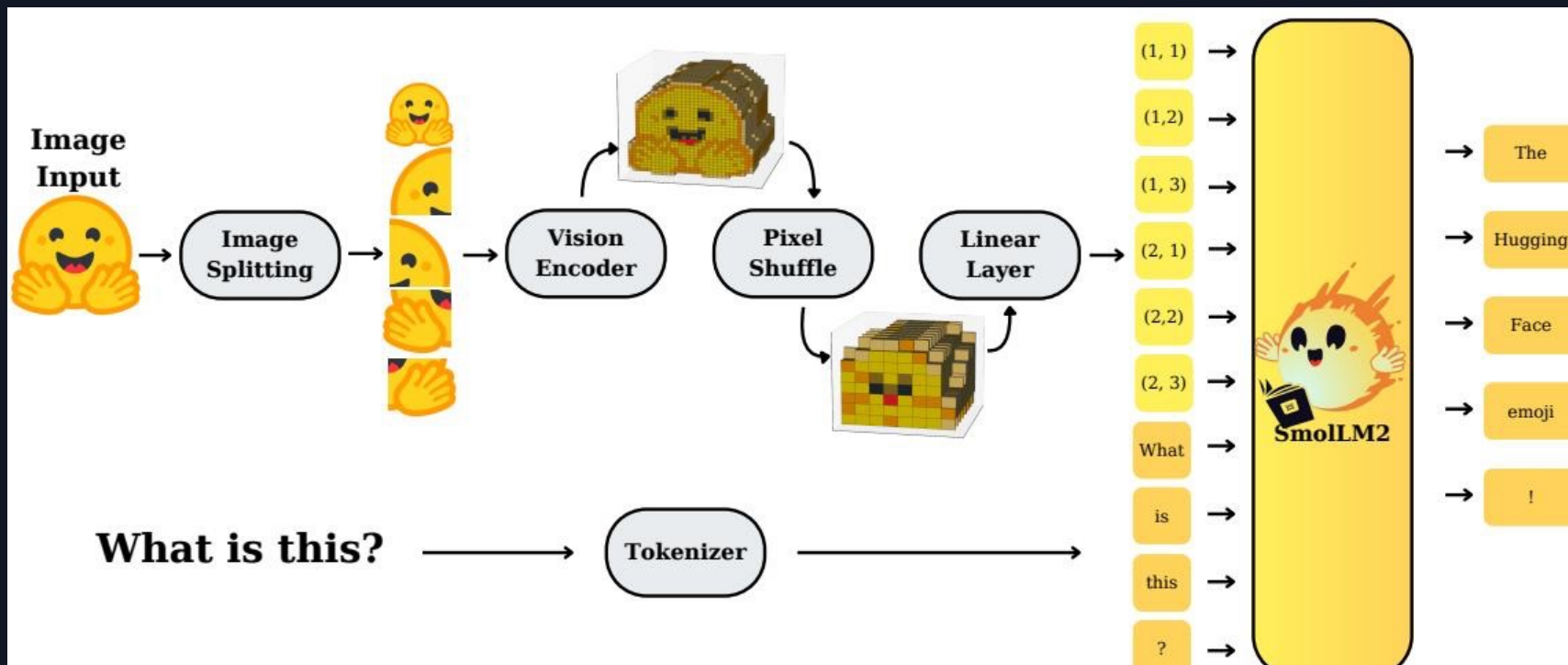
Goal: retain multimodal capability with extreme memory efficiency

RAM at batch=1 (from paper)



SmolVLM Architecture: Image Splitting, Pixel Shuffle, and SmolLM2 Backbone

Pipeline



Flow (3-4 steps)

Split image → encode sub-images with SigLIP vision encoder

Pixel shuffle (space-to-depth) compresses visual tokens

Architecture

Linear projection maps visual features into SmolLM2 embedding space

Image → tokens → SmolLM2

Smaller Vision Encoders Outperform Larger Ones in Compact VLMs

Finding 1 — Balanced encoder/LM compute

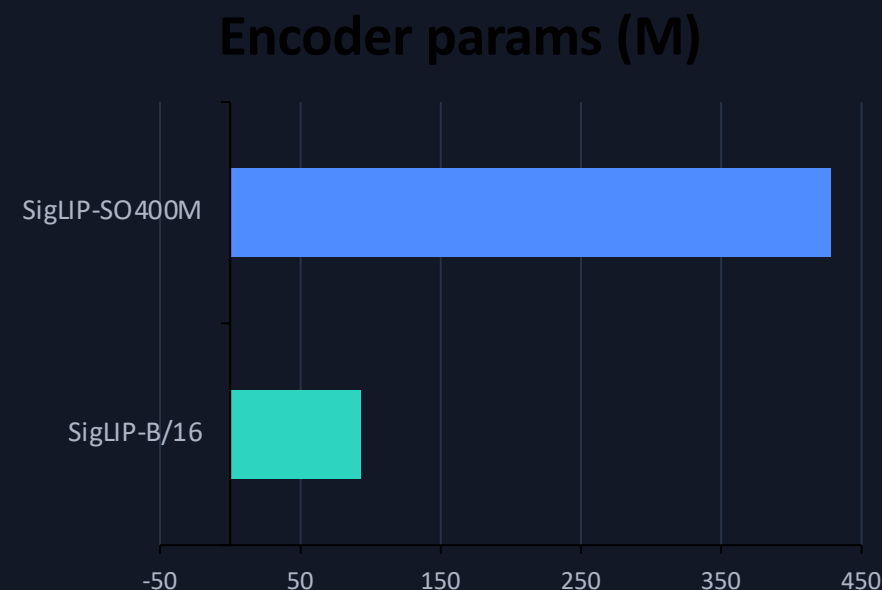
For compact LMs, allocating too many params to the encoder can hurt

93M SigLIP-B/16 + 135M LM > 428M SigLIP-SO400M + same LM

Only at ~1.7B LM scale does the larger encoder pay off

Takeaway: scale encoder with LM, not independently

Comparison visual (qualitative)



Small-LM regime: smaller encoder wins

(93M SigLIP-B/16 outperforms 428M SigLIP-SO400M when paired with 135M LM)

Aggressive Pixel Shuffle (r=4) Cuts Visual Tokens 16× for Small Models

Finding 3 — Token compression

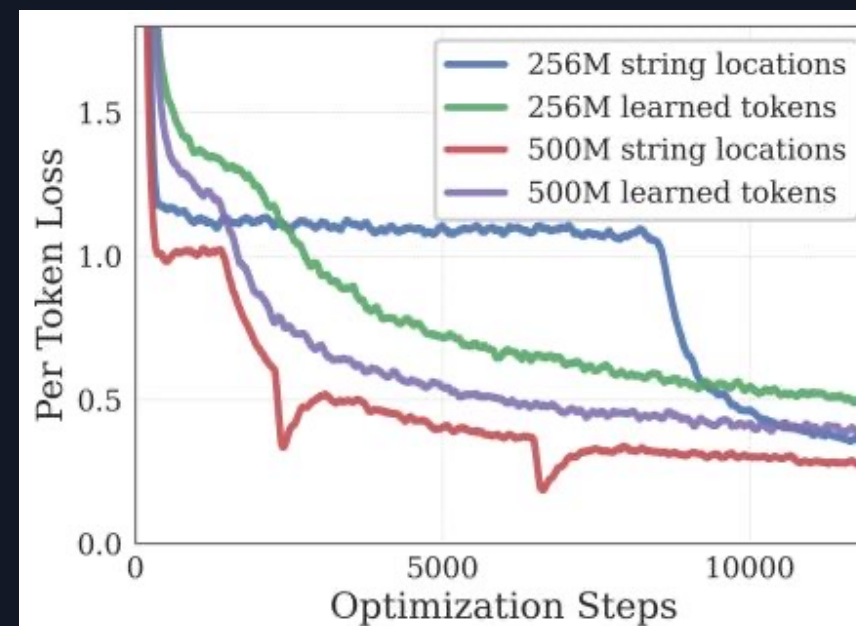
Pixel shuffle (space-to-depth) rearranges H×W into channels

Token count scales down by r^2 ($r=4 \Rightarrow 16\times$ fewer visual tokens)

Small VLMs benefit from $r=4$; larger models prefer $r=2$

Net effect: lower attention cost + better long-context behavior

Pixel shuffle mechanism



$r = 4 \rightarrow 16\times$ fewer tokens

Extending Context from 2K to 16K Tokens Unlocks Higher Image Resolution

Finding 2 — Extended context length

Performance improves significantly when extending context 2K → 16K tokens

Method: increase RoPE base 10K → 273K + fine-tune on mixed long/short-context data

SmolVLM-2.2B uses 16K context; 256M/500M use 8K

Long context supports higher effective image resolution and more frames/text

Context scaling

2K

8K

16K

RoPE base: 10K → 273K

SmolVLM Achieves Competitive Performance at Sub-1GB to 4.9GB RAM

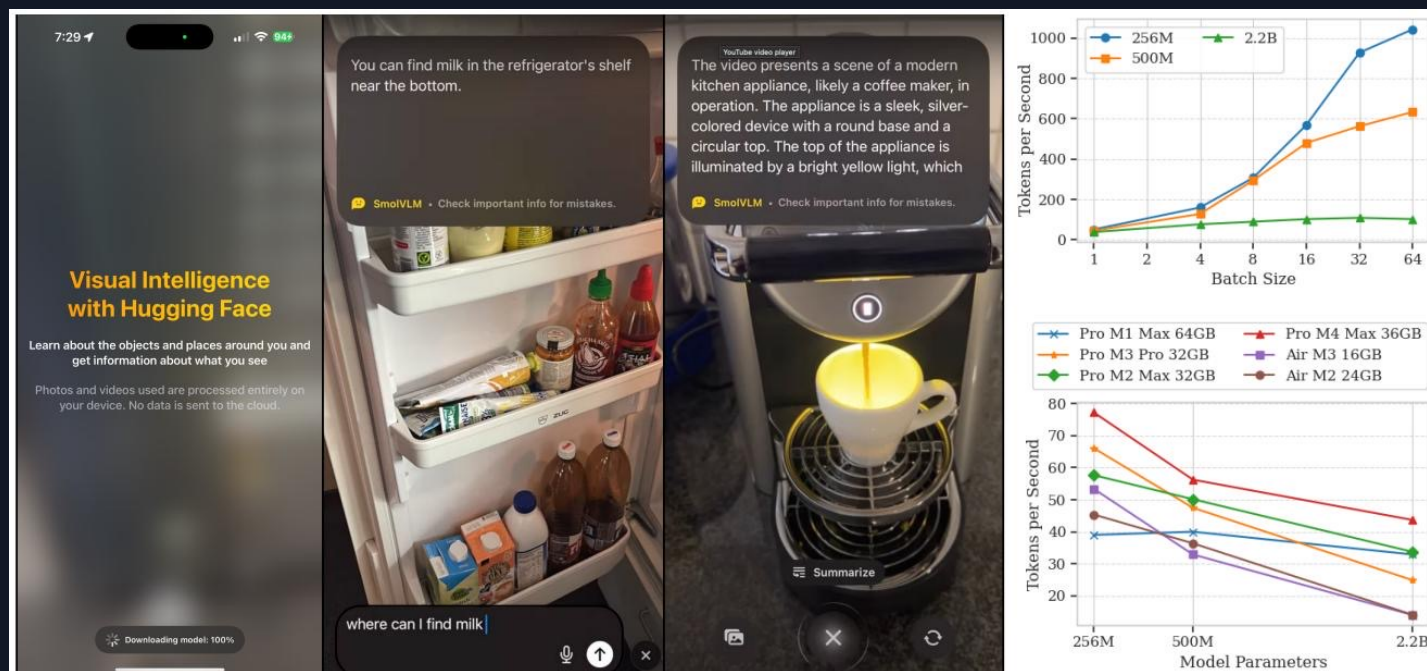
13-benchmark summary + memory

Capability	Benchmark	SmolVLM 256M	SmolVLM 500M	SmolVLM 2.2B	Best Efficient OS
Single-Image	OCRBench	52.6%	61.0%	72.9%	54.7% (Molmo-A1B-7B)
Single-Image	TextVQA	48.0%	50.2%	70.0%	70.0% (Molmo-A1B-7B)
Single-Image	ChartQA	55.6%	62.8%	68.7%	48.0% (Molmo-A1B-7B)
Single-Image	TextVQA	50.0%	60.0%	78.0%	60.0% (Molmo-A1B-7B)
Single-Image	DocVQA	58.3%	70.5%	80.0%	77.7% (Molmo-A1B-7B)
Single-Image	ScienceQA	73.0%	81.0%	88.0%	87.5% (Molmo-A1B-7B)
Multi-task	MMMU	29.0%	33.7%	42.0%	33.9% (Molmo-A1B-7B)
Multi-task	MMVista	35.0%	40.0%	50.0%	47.0% (Molmo-A1B-7B)
Multi-task	MMStar	34.6%	38.3%	46.0%	43.1% (Molmo-A1B-7B)
Video	Video-MME	33.2%	42.2%	58.0%	45.0% (InternVL2-2B)
Video	MLVU	40.6%	47.3%	55.2%	48.2% (InternVL2-2B)
Average	All Benchmarks	44.0%	51.0%	59.8%	—

RAM (batch=1): 256M 0.8GB • 500M 1.2GB • 2.2B 4.9GB vs Molmo-A1B-7B 27.7GB

Real-Time Inference on iPhones: Up to 80 Tokens/Second on Apple Silicon

HuggingSnap on-device deployment



Deployment metrics (from paper)

On-device photo + video understanding (no cloud)

SmolVLM-256M: up to ~80 tokens/s on Pro M1 Max 64GB

Batch=64: nearly 1000 tokens/s on high-end devices

MacBook Air M3 16GB: ~55 tokens/s (256M)

Throughput scales roughly linearly with batch for 256M

Key Takeaways: Design Principles for Efficient Multimodal Models

Actionable principles

Balance encoder–LM compute: smaller encoders can win in small-LM regimes

Use aggressive token compression (pixel shuffle): $r=4$ helps small VLMs

Treat context length as a capability lever: 2K→16K via RoPE base + mixed-context tuning

Memory-first targets are feasible: 0.8GB (256M), 1.2GB (500M), 4.9GB (2.2B) at batch=1

Open release (weights + data + code + app) enables reproducible on-device research

Design for tokens, context, and memory — then scale parameters.